

William Jackson

Senior Software Engineer – Python(Django/FastAPI/Flask) AWS

Wellington, Nevada, United States | +1 (786) 949-7561 | williamjackson.pro@outlook.com | linkedin.com/in/william-jackson-75b9343b6

PROFESSIONAL SUMMARY

Senior Software Engineer with ~10 years building **Python backend systems** for **network operations, data products, operational SaaS, and reliability-focused business platforms** where stable APIs, trustworthy data, and fast incident recovery directly affect customer confidence. Strong hands-on depth across **Python 3.8–3.12, Django 2.2–5.x, FastAPI 0.7x–0.11x, and Flask 1.1–3.x**, designing services with clean boundaries, durable contracts, and pragmatic security controls. Known for modernizing fragile legacy services, improving API correctness, reducing alert noise, and delivering measurable reliability gains through **AWS, CI/CD, observability, and disciplined post-incident hardening.**

SKILLS

- **Languages & Backend Frameworks** — **Python 3.8–3.12, Django 2.2–5.x, FastAPI 0.7x–0.11x, Flask 1.1–3.x**, SQL, Bash, backend architecture, service layering, middleware design, background processing, admin tooling, ORM patterns, schema validation
- **Backend & APIs** — **REST APIs**, GraphQL, microservices, event-driven architecture, OpenAPI, WebSockets, background jobs, pagination, filtering, API versioning, idempotency, rate limiting, retries, circuit breakers, caching strategies, file-processing workflows, worker patterns
- **Data, Messaging & Search** — PostgreSQL, MySQL, SQL Server, Redis, MongoDB, DynamoDB, AWS SQS/SNS, Kafka, RabbitMQ, Elasticsearch, OpenSearch, Redshift, BigQuery
- **Cloud & DevOps** — **AWS** (EKS, ECS, EC2, Lambda, API Gateway, S3, CloudFront, RDS, DynamoDB, ElastiCache, EventBridge, Step Functions, IAM), **Azure** (AKS, App Service, Azure SQL, Service Bus, Key Vault, Application Insights), **GCP** (GKE, Cloud Run, Pub/Sub, Cloud Storage, BigQuery), Docker, Kubernetes, Helm, Argo CD, GitOps, Terraform, CloudFormation, GitHub Actions, GitLab CI, Jenkins, Azure DevOps, CircleCI
- **Observability, Quality & Security** — OpenTelemetry, Datadog, Prometheus, Grafana, ELK/EFK, CloudWatch, structured logging, Jest, Mocha, Chai, Cypress, Playwright, Pact-style contract testing, OAuth2, OpenID Connect, JWT, Okta, Auth0, AWS Cognito, RBAC, ABAC, audit logging

WORK HISTORY

Senior Software Engineer

February 2021 to December 2025

Viaro Networks Inc. - Delaware, United States

- Architected backend platforms using **Python 3.10–3.12, FastAPI**, and **Django**, turning loosely defined operational requirements into stable service contracts and measurable releases for reliability-focused internal and customer-facing workflows.
- Led migration of older **Flask** and Django-style modules into **FastAPI** services for high-throughput APIs, reducing validation drift by standardizing request schemas with **Pydantic** and improving maintainability across teams.
- Designed **Django REST Framework** and **FastAPI** endpoints with backward-compatible pagination, filtering, error contracts, and auth boundaries so downstream clients could adopt new features without frequent breaking changes.
- Resolved a recurring production issue where malformed payloads triggered inconsistent 500-series failures by introducing typed validation, centralized exception handling, and correlation-aware structured logs, which significantly reduced noisy incidents.
- Built async processing workflows with **Celery, SQS/SNS, Kafka**, and **RabbitMQ**, implementing idempotency keys, retry policies, and dead-letter handling so replayed jobs stayed safe during partial failures and recovery events.
- Improved API latency and read performance by combining **Redis/ElastiCache** caching, query tuning, and smarter serialization paths, cutting pressure on relational stores during spiky operational traffic.
- Modernized data access across **PostgreSQL, MySQL, SQL Server, MongoDB**, and **DynamoDB**, introducing stronger indexing, schema discipline, and safer transaction boundaries to stabilize high-volume backend workloads.
- Implemented secure authentication and authorization flows using **OAuth2, OpenID Connect, JWT**, and **AWS Cognito**, while enforcing **RBAC/ABAC** and audit logging for sensitive administrative and customer-impacting actions.

- Owned observability rollout using **OpenTelemetry**, **Datadog**, **CloudWatch**, **Prometheus**, and **Grafana**, replacing ad hoc debugging with traceable request lifecycles and SLO-aware alerting that improved root-cause isolation.
- Delivered workflow orchestration with **EventBridge** and **Step Functions** for long-running backend tasks, allowing multi-step processing to survive retries, restarts, and partial dependency outages without corrupting state.
- Deployed services across **EKS**, **ECS**, **EC2**, and **Lambda** using **Docker**, **Kubernetes**, **Helm**, and repeatable release patterns, making backend delivery safer and more predictable across multiple environments.
- Reduced deployment risk by codifying infrastructure in **Terraform** and **CloudFormation**, which made environment drift visible during review and helped backend teams reproduce changes consistently across staging and production.
- Strengthened quality gates with **Pytest**, contract tests, integration tests, and browser-driven validation where needed, catching schema mismatches and backend regressions before they reached customer workflows.
- Consolidated logging and failure analysis through **ELK/EFK**, trace IDs, and structured event context, turning incident response into evidence-based debugging instead of fragmented guesswork across distributed services.
- Partnered closely with product, ops, and platform teams to ship new backend capabilities incrementally, balancing speed with operability and ensuring that each release improved reliability rather than adding hidden support costs.

Software Engineer
Avantia, Inc. - Ohio, United States

October 2018 to January 2021

- Built backend data and API services using **Python 3.8–3.10**, **Django**, **Flask**, and early **FastAPI** adoption for internal analytics and operational processing workflows that required stable contracts and reliable execution.
- Implemented reproducible preprocessing and transformation pipelines that converted raw operational inputs into clean backend-ready datasets, helping teams trust reruns, backfills, and downstream reporting outputs.
- Refactored brittle ETL-heavy backend logic into smaller composable services and reusable modules, reducing failure blast radius and making targeted fixes faster when partial job chains broke in production.
- Resolved an issue where inconsistent serializer behavior caused client-facing data mismatches by introducing stronger schema validation, explicit normalization rules, and regression coverage around critical payload shapes.
- Introduced queue-backed worker execution using **SQS/SNS** and scheduled processing patterns, allowing long-running backend jobs to recover safely from timeouts, spikes, and partial dependency failures.
- Improved batch performance by profiling hot paths and applying vectorized operations, chunked processing, and more efficient ORM/database usage, keeping processing windows predictable as data volume increased.
- Implemented pragmatic **Redis** caching for repeated lookups and computed backend responses, reducing unnecessary database round-trips while preserving freshness through clear TTL and invalidation rules.
- Supported secrets management and environment hardening with IAM-aligned patterns and managed credential workflows, including exposure to **Azure Key Vault** style implementations for secure backend configuration.
- Extended integrations into **Azure** and **GCP** destinations, including export paths to storage and analytics systems such as **BigQuery**, giving backend services broader interoperability across client environments.
- Built metrics-first monitoring and dashboards with **Prometheus**, **Grafana**, and **CloudWatch** patterns so backend failures became visible early and recovery steps could be based on signals instead of guesswork.
- Standardized local development and CI behavior with **Docker**, **GitHub Actions**, and **Jenkins**, reducing “works on my machine” drift and making backend changes easier to verify before release.
- Added integration and contract coverage around data transforms, API responses, and export routines, preventing silent schema drift from breaking dependent consumers after otherwise small code changes.
- Worked directly with stakeholders through demos, acceptance checks, and incremental delivery, ensuring backend changes stayed aligned with business needs while keeping implementation choices practical and supportable.

Software Developer
ArnAmy, Inc. - Florida, United States

March 2016 to September 2018

- Developed backend data-processing and service components using **Python 3.6–3.8**, **Flask 0.12–1.x**, and **Django 1.11–2.2** patterns to normalize operational records and expose curated datasets for internal products.
- Built service endpoints with defensive pagination, filtering, and payload limits so APIs remained stable under growing usage and did not overload relational systems during bursty access patterns.

- Implemented reconciliation routines that compared source-of-truth feeds with derived backend tables, surfacing mismatches as actionable reports instead of letting silent data divergence accumulate over time.
- Automated recurring backfills and validation routines with duplicate-ingestion guardrails, which prevented accidental double-processing and protected aggregate correctness during reruns and operational recovery work.
- Improved performance across **PostgreSQL**, **MySQL**, and **SQL Server** through index tuning, query rewrites, and more careful ORM usage, producing more stable response times as dataset volume expanded.
- Resolved recurring failures in long-running jobs by adding structured logs, richer failure context, and better retry boundaries, making backend issues diagnosable without needing debugger-only reproduction paths.
- Stabilized third-party integrations by introducing timeouts, bounded retries, and safer request limits, preventing upstream slowness from cascading into prolonged backend outages and worker pileups.
- Introduced **Docker**-based development and deployment hygiene for backend services, reducing environment drift and making local verification more consistent across developers and test environments.
- Supported search-oriented workflows by generating export-ready datasets compatible with **Elasticsearch** and **OpenSearch** indexing patterns, improving discoverability for operational and reporting use cases.
- Implemented **Redis** caching for read-heavy backend paths, defining key composition and expiration strategies carefully so cached responses improved speed without undermining correctness or freshness expectations.
- Partnered with product and operations stakeholders to shape backend work into incremental deliverables, improving release confidence and reducing confusion around loosely defined feature requests.

Junior Software Developer
Centric Tech Inc. - New Jersey, United States

October 2014 to February 2016

- Delivered early backend utilities and processing scripts using **Python 2.7–3.x** and lightweight **Flask** patterns, helping teams clean, validate, and reconcile messy operational datasets more consistently than spreadsheet-driven workflows.
- Built internal tools to standardize identifiers, timestamp handling, and input normalization, which reduced downstream mismatches and made reporting outputs more trustworthy across teams.
- Improved backend reliability by adding defensive parsing, clearer validation messages, and safer fallbacks so bad data surfaced early instead of causing silent corruption or vague support issues later.
- Assisted with service integrations by introducing timeout controls and safe retry behavior, reducing failures caused by transient network instability and improving baseline resilience for connected systems.
- Supported release validation by comparing backend outputs against known-good samples, sanity thresholds, and historical patterns, helping catch regressions before they spread into broader operational workflows.
- Contributed to early CI discipline through scripted checks and lightweight pipeline steps, making backend changes more repeatable across environments and reducing manual verification effort during handoffs.
- Helped troubleshoot production issues through structured logs, SQL inspection, and small reproducible tests, building strong habits around bottleneck isolation and practical root-cause analysis.
- Documented expected inputs, outputs, and failure modes for small backend jobs and utilities, which made maintenance easier and allowed later enhancements to happen with less uncertainty.
- Collaborated closely with senior engineers to convert vague requests into concrete backend tasks, gradually expanding ownership while learning how stable APIs and data contracts are shaped in real systems.

EDUCATION

Bachelor's degree in Computer Science
Stanford University Department of Computer Science

2010 to 2014

- Focused on strong CS fundamentals and applied systems work, building a durable base for **distributed services**, **data modeling**, and production engineering practices.